



Introduction to Java Programming

& VS Code Setup

Instructor: Khurram Iqbal



Learn Java Fundamentals

From basic syntax to creating your first program



Set Up Your Development Environment

Configure VS Code for Java development

What is Java?

Definition

Java is a **high-level, class-based, object-oriented** programming language designed to have as few implementation dependencies as possible.

Created by **James Gosling** at Sun Microsystems (now owned by Oracle) and released in 1995.



Philosophy: "Write Once, Run Anywhere"

Java code is compiled to bytecode that runs on the Java Virtual Machine (JVM), allowing it to run on any device with a JVM installed.



This platform independence makes Java ideal for cross-platform applications and enterprise systems.

Key Features of Java

- Object-Oriented**

Everything in Java is an object, making code modular, flexible, and extensible.
- Platform Independent**

Java programs can run on any device with a JVM installed.
- Simple & Familiar Syntax**

Designed to be easy to learn, especially for those familiar with C/C++.
- Robust & Secure**

Strong memory management, exception handling, and security features.
- Multithreaded**

Built-in support for concurrent programming and parallel execution.

Why Learn Java?

- ✔ One of the most widely used programming languages in the world
- ✔ High demand in job market with excellent career opportunities
- ✔ Versatile: used for web, mobile, desktop, and enterprise applications
- ✔ Large community and extensive libraries/frameworks
- ✔ Excellent foundation for learning other programming languages

Common Java Applications



Web Applications
Spring, JSP, Servlets



Mobile Apps
Android Development



Desktop Apps
JavaFX, Swing



Enterprise Systems
Banking, Finance, Big Data

Java Platform Components

The Java Platform Architecture

The Java platform consists of three main components that work together to create and run Java applications:



How They Work Together

Development Process

1. Write Java source code (.java files)
2. Compile code using `javac` compiler (part of JDK)
3. Compiler creates bytecode (.class files)
4. JVM executes the bytecode



The JVM is what enables Java's "Write Once, Run Anywhere" capability by providing a consistent runtime environment across different platforms.

Java Development Kit (JDK)

The JDK is a software development environment used for developing Java applications.

Key Components

- ✔ **Java Compiler (javac):** Converts .java files to .class files
- ✔ **Development Tools:** Debugger, documentation tools, etc.
- ✔ **JRE:** Complete runtime environment

The JDK is what developers install to write and compile Java code.

Java Runtime Environment (JRE)

The JRE provides the libraries, Java Virtual Machine, and other components needed to run Java applications.

Key Components

- ✔ **Java Virtual Machine (JVM):** Executes Java bytecode
- ✔ **Java Class Libraries:** Standard APIs and packages
- ✔ **Integration Libraries:** For browser plugins, etc.

End users who only want to run Java applications need only the JRE, not the full JDK.

Java Virtual Machine (JVM)

The JVM is an abstract computing machine that enables a computer to run Java programs.

Key Functions

- ✔ **Loads and verifies** class files
- ✔ **Interprets bytecode** into machine code
- ✔ Manages **memory allocation** and garbage collection
- ✔ Provides **security** and isolation

The JVM is platform-specific but provides a consistent environment for Java bytecode.

JDK Versions

Standard Edition (SE)

Core platform for desktop and server applications. Contains the essential libraries and APIs.

Enterprise Edition (EE)

Built on SE, adds libraries for large-scale, multi-tiered, scalable, reliable, and secure network applications.

Micro Edition (ME)

For developing applications for mobile devices and embedded systems with limited resources.



For beginners, we'll focus on Java SE (Standard Edition), which is what you'll need for learning the fundamentals of Java programming.

Installing the Java Development Kit (JDK)

Before You Begin

Which JDK Version?

For beginners, we recommend:

- ✔ **Java SE Development Kit (JDK) 17 LTS** - Long Term Support version
- ✔ Or **JDK 21 LTS** - Latest Long Term Support version

JDK Distributions

Several vendors provide JDK distributions:

- ✔ **Oracle JDK** - Official Oracle release
- ✔ **OpenJDK** - Free, open-source implementation
- ✔ **Amazon Corretto**, **AdoptOpenJDK**, etc.

For this tutorial, we'll use Oracle's OpenJDK.

Windows Installation

Step-by-Step Guide

1. Download the Windows .exe or .msi installer
2. Run the installer with administrator privileges
3. Follow the installation wizard prompts
4. Accept the default installation location or choose your own
5. Wait for installation to complete

Setting Path Variables (if needed)

1. Right-click on "This PC" → Properties → Advanced system settings
2. Click "Environment Variables"
3. Under System variables, click "New" and add JAVA_HOME
4. Set value to JDK installation path (e.g., C:\Program Files\Java\jdk-17)
5. Edit "Path" variable and add %JAVA_HOME%\bin



Modern JDK installers for Windows typically set the PATH variable automatically, so manual configuration may not be necessary.

General Installation Steps

Step 1: Download JDK

Visit <https://jdk.java.net/> or <https://www.oracle.com/java/technologies/downloads/>

Step 2: Run the Installer

Follow the installation wizard instructions for your OS

Step 3: Set JAVA_HOME (if needed)

Configure environment variable to point to JDK installation directory

Step 4: Verify Installation

Open terminal/command prompt and run: `java -version`

macOS Installation

Step-by-Step Guide

1. Download the macOS .dmg or .pkg installer
2. Open the downloaded file
3. Follow the installation wizard prompts
4. Enter your administrator password when prompted
5. Wait for installation to complete

Setting Path Variables (if needed)

1. Open Terminal
2. Edit your shell profile file (~/.bash_profile, ~/.zshrc, etc.)
3. Add: `export JAVA_HOME=$(/usr/libexec/java_home)`
4. Add: `export PATH=$JAVA_HOME/bin:$PATH`
5. Save and run: `source ~/.bash_profile` (or your profile file)

Linux Installation

Using Package Manager (Ubuntu/Debian)

```
sudo apt update
sudo apt install openjdk-17-jdk
```

Manual Installation

1. Download the Linux .tar.gz archive
2. Extract to desired location: `tar -xvf jdk-17_linux-x64_bin.tar.gz`
3. Move to /opt: `sudo mv jdk-17 /opt/`

Setting Path Variables

1. Edit ~/.bashrc or ~/.profile
2. Add: `export JAVA_HOME=/opt/jdk-17`
3. Add: `export PATH=$JAVA_HOME/bin:$PATH`
4. Save and run: `source ~/.bashrc`

Verifying Your Installation

Check Java Version

Open a terminal or command prompt and run:

```
java -version
javac -version
```

You should see output showing the installed Java version.

Sample Output

```
$ java -version
openjdk version "17.0.2" 2022-01-18
OpenJDK Runtime Environment (build 17.0.2+8-86)
OpenJDK 64-Bit Server VM (build 17.0.2+8-86, mixed mode, sharing)

$ javac -version
javac 17.0.2
```



If you receive a "command not found" error, your installation or PATH configuration may be incorrect. Double-check your installation and environment variables.

Introduction to Visual Studio Code

What is VS Code?

Visual Studio Code (VS Code) is a lightweight but powerful source code editor developed by Microsoft for Windows, macOS, and Linux.

It's **free**, **open-source**, and has become one of the most popular code editors among developers.



Key Features for Java Development

</> Syntax Highlighting

Color-coded syntax makes code easier to read and understand

💡 IntelliSense

Code completion, parameter info, quick info, and member lists

🐛 Debugging

Set breakpoints, step through code, inspect variables

>_ Integrated Terminal

Run commands without leaving the editor

🔑 Git Integration

Built-in Git commands and source control

🧩 Extensions

Customize and enhance functionality with extensions

Why VS Code for Java Development?

Lightweight & Fast

Starts quickly and remains responsive even with large projects

Excellent Java Support

Through extensions that provide IDE-like features

Cross-Platform

Works consistently across Windows, macOS, and Linux

Free & Open Source

No licensing costs or restrictions

VS Code Interface

Main Components

- ✔ **Activity Bar:** Switch between different views
- ✔ **Side Bar:** Explorer, Search, Source Control, Run, Extensions
- ✔ **Editor:** Main area for editing files
- ✔ **Status Bar:** Information about the current project and file
- ✔ **Panel:** Terminal, Output, Problems, Debug Console

💡 VS Code's interface is highly customizable. You can rearrange panels, change themes, and adjust settings to create your ideal development environment.

VS Code vs. Other Java IDEs

VS Code

- + Lightweight and fast
- + Highly customizable
- + Great for multi-language projects
- Requires extensions for full Java support

Eclipse

- + Comprehensive Java tooling built-in
- + Mature ecosystem
- Heavier resource usage
- Steeper learning curve

IntelliJ IDEA

- + Advanced code analysis
- + Powerful refactoring tools
- Resource intensive
- Full version requires paid license



For beginners, VS Code offers an excellent balance of simplicity, performance, and features. With the right extensions, it provides all the functionality needed for Java development while remaining lightweight and easy to use.

Installing Visual Studio Code

Before You Begin

System Requirements

- ✔ **Windows:** Windows 8, 10, 11 (64-bit)
- ✔ **macOS:** macOS 10.11+
- ✔ **Linux:** Debian, Ubuntu, Red Hat, Fedora, SUSE



💡 VS Code is lightweight but having at least 4GB of RAM and a modern processor will ensure a smooth experience, especially when working with larger projects.

Windows Installation

Step-by-Step Guide

1. Download the Windows installer (.exe file)
2. Run the installer
3. Accept the license agreement
4. Choose installation location (default is recommended)
5. Select Start Menu folder
6. Select additional tasks:
 - Add "Open with Code" to context menu
 - Add to PATH (recommended)
7. Click Install and wait for completion
8. Launch VS Code

General Installation Steps

Step 1: Download VS Code

Visit <https://code.visualstudio.com/download> and select your platform

Step 2: Run the Installer

Follow the installation wizard instructions for your OS

Step 3: Launch VS Code

Open VS Code from your applications menu or desktop shortcut

Step 4: Install Extensions

Add Java extensions (covered in the next slide)

macOS Installation

Step-by-Step Guide

1. Download the macOS .zip file
2. Extract the zip file
3. Drag Visual Studio Code.app to the Applications folder
4. Launch VS Code from Applications folder or Spotlight
5. Optional: Add VS Code to Dock

Adding to PATH (Optional)

To enable the 'code' command in terminal:

1. Launch VS Code
2. Open Command Palette (⌘+Shift+P)
3. Type "shell command" and select "Install 'code' command in PATH"

Linux Installation

Using Package Manager

Debian/Ubuntu

```
# Download the package
wget -qO-
https://packages.microsoft.com/keys/microsoft.asc |
gpg --dearmor > packages.microsoft.gpg
sudo install -o root -g root -m 644
packages.microsoft.gpg /etc/apt/trusted.gpg.d/
sudo sh -c 'echo "deb [arch=amd64,arm64,armhf signed-
by=/etc/apt/trusted.gpg.d/packages.microsoft.gpg]
https://packages.microsoft.com/repos/code stable main"
> /etc/apt/sources.list.d/vscode.list'
sudo apt install apt-transport-https
sudo apt update
sudo apt install code
```

RHEL/Fedora/CentOS

```
sudo rpm --import
https://packages.microsoft.com/keys/microsoft.asc
sudo sh -c 'echo -e "[code]\nname=Visual Studio
Code\nbaseurl=https://packages.microsoft.com/yumrepos/vscode\nenable
> /etc/yum.repos.d/vscode.repo'
sudo dnf install code
```

Using .deb or .rpm Package

1. Download the .deb or .rpm package from the VS Code website
2. Install using your package manager
3. For .deb: `sudo apt install ./code_*.deb`
4. For .rpm: `sudo rpm -i code-*.rpm`

Verifying Your Installation

Launch VS Code

After installation, launch VS Code:

- ✔ **Windows:** Start menu or desktop shortcut
- ✔ **macOS:** Applications folder or Dock
- ✔ **Linux:** Applications menu or terminal with code command

Check Version

To verify the installation and check the version:

1. Launch VS Code
2. Click on Help → About (Windows/Linux) or Code → About Visual Studio Code (macOS)
3. Or use terminal/command prompt: `code --version`

⚠ If VS Code doesn't launch or you encounter errors, try reinstalling or check the [official documentation](#) for troubleshooting steps.

ℹ Now that you have VS Code installed, the next step is to install the Java extensions to enable Java development features.

Installing Java Extensions in VS Code

Why Extensions?

VS Code is a lightweight editor by default. To enable Java development features, you need to install Java-specific extensions.

Extensions Add:

- ✔ Java language support and syntax highlighting
- ✔ Code completion and IntelliSense
- ✔ Debugging capabilities
- ✔ Project management tools
- ✔ Testing framework integration



With the right extensions, VS Code transforms from a simple text editor into a powerful Java IDE!

How to Install Extensions

Method 1: Extensions View

- Click on the Extensions icon in the Activity Bar (or press Ctrl+Shift+X / Cmd+Shift+X)
- Search for the extension name
- Click the "Install" button

Method 2: Command Palette

- Open Command Palette (Ctrl+Shift+P / Cmd+Shift+P)
- Type "Extensions: Install Extensions"
- Search and install desired extensions

Method 3: Extension Pack

- Install "Extension Pack for Java" which includes multiple Java extensions
- This is the recommended approach for beginners

Recommended Java Extensions

Extension Pack for Java

A collection of popular extensions for Java development:

- Language Support for Java by Red Hat
- Debugger for Java
- Test Runner for Java
- Maven for Java
- Project Manager for Java
- Visual Studio IntelliCode

Publisher: Microsoft

Language Support for Java™ by Red Hat

Provides Java language support via Eclipse JDT Language Server. Includes:

- IntelliSense
- Code navigation
- Refactoring
- Code snippets

Publisher: Red Hat

Debugger for Java

A lightweight Java debugger based on Java Debug Server:

- Launch/Attach
- Breakpoints
- Exceptions
- Variables & Watch
- Call Stacks

Publisher: Microsoft

Additional Useful Extensions

Maven for Java

Project management and build automation

Test Runner for Java

Run and debug JUnit/TestNG tests

Java Dependency Viewer

View Java dependencies in a tree

Visual Studio IntelliCode

AI-assisted code completions

Checkstyle for Java

Code style checking and formatting

SonarLint

Code quality and security analysis



For beginners, start with just the "Extension Pack for Java" which includes the most essential extensions. You can add more as needed later.

Installing the Extension Pack for Java

Step 1



Open VS Code and click on the Extensions icon in the Activity Bar (or press Ctrl+Shift+X / Cmd+Shift+X)

Step 2



Search for "Extension Pack for Java" in the search bar

Step 3



Click the "Install" button on the Extension Pack for Java by Microsoft

After Installation

- ✔ VS Code will download and install all included extensions
- ✔ You may need to reload VS Code when prompted
- ✔ VS Code will automatically detect your JDK installation
- ✔ Java Language Server will initialize in the background
- ✔ A notification will appear when setup is complete
- ✔ You're now ready to configure VS Code for Java development!

Configuring VS Code for Java

JDK Configuration

After installing the Java extensions, VS Code needs to know where your JDK is located.

Automatic Detection

VS Code will attempt to automatically detect your JDK installation. If successful, no further action is needed.

Manual Configuration

If automatic detection fails, you can manually set the JDK path:

- Open Command Palette (Ctrl+Shift+P / Cmd+Shift+P)
- Type "Java: Configure Java Runtime"
- Click "Configure Java Runtime"
- Under "JDK Installations", click "Add"
- Browse to your JDK installation folder

Recommended Settings

Example settings.json

```
{
  "java.home": "/path/to/jdk",
  "java.project.sourcePaths": ["src"],
  "java.project.outputPath": "bin",
  "java.format.settings.url": "eclipse-formatter.xml",
  "java.format.settings.profile": "Eclipse",
  "editor.formatOnSave": true,
  "java.completion.importOrder": [
    "java",
    "javax",
    "com",
    "org"
  ]
}
```

Key Settings Explained

-  **java.home:** Path to JDK installation
-  **java.project.sourcePaths:** Source code directories
-  **java.project.outputPath:** Compiled class files directory
-  **editor.formatOnSave:** Format code when saving

Workspace Settings

Creating a Workspace

For better organization, create a dedicated workspace for Java projects:

- Create a new folder for your Java projects
- In VS Code, go to File → Open Folder
- Select your Java projects folder
- Go to File → Save Workspace As...
- Name your workspace (e.g., "JavaProjects.code-workspace")

Workspace Settings File

Create or modify settings.json for Java-specific configurations:

- Open Command Palette (Ctrl+Shift+P / Cmd+Shift+P)
- Type "Preferences: Open Workspace Settings (JSON)"
- Add Java-specific settings (examples in next section)

Editor Preferences

Font & Theme

Customize your coding environment:

- File → Preferences → Settings
- Search for "font" or "theme"
- Recommended fonts: Fira Code, JetBrains Mono
- Enable font ligatures for better readability

Code Formatting

Set up automatic code formatting:

- File → Preferences → Settings
- Search for "format"
- Enable "Editor: Format On Save"
- Configure "Java > Format: Settings"

IntelliSense

Customize code completion behavior:

- File → Preferences → Settings
- Search for "suggest"
- Adjust "Editor: Quick Suggestions"
- Configure "Editor: Suggest Selection"

Key Bindings

Customize keyboard shortcuts:

- File → Preferences → Keyboard Shortcuts
- Search for Java-related commands
- Click the pencil icon to edit shortcuts
- Consider Eclipse or IntelliJ keymaps

Verifying Configuration

Check Java Environment

- Open Command Palette (Ctrl+Shift+P / Cmd+Shift+P)
- Type "Java: Configure Java Runtime"
- Verify that your JDK is listed and selected
- Check that the version matches what you installed



If you see any warnings or errors in the "Problems" panel related to Java configuration, address them before proceeding to create your first Java project.

Troubleshooting Common Issues

JDK Not Found

-  Manually set java.home in settings.json
-  Verify JAVA_HOME environment variable
-  Restart VS Code after changes

Extension Issues

-  Check extension requirements
-  Disable and re-enable extensions
-  Update to latest versions

Language Server

-  Restart Java Language Server
-  Check Output panel for errors
-  Clean workspace cache



Now that you have VS Code configured for Java development, you're ready to create your first Java project and start coding!

Creating a Java Project in VS Code

Project Types in VS Code

Simple Java Project

- ✔ Basic folder structure with Java files
- ✔ Suitable for beginners and small programs
- ✔ No build tools or complex configuration

Maven/Gradle Projects

- ✔ Build automation and dependency management
- ✔ Standard project structure
- ✔ Better for larger projects and teams

For this tutorial, we'll focus on creating a simple Java project.

Using Project Creation Commands

Java: Create Java Project

- Open Command Palette (Ctrl+Shift+P / Cmd+Shift+P)
- Type "Java: Create Java Project"
- Select project type:
 - No build tools
 - Maven
 - Gradle
 - Spring Boot
- Follow the prompts to complete project creation



For beginners, select "No build tools" to create a simple Java project without the complexity of build systems.

Creating a Simple Java Project

Step 1: Create Project Folder

- Open VS Code
- Go to File → Open Folder (or Ctrl+K Ctrl+O / Cmd+O)
- Create a new folder for your project (e.g., "MyFirstJavaProject")
- Select the folder and click "Open"

Step 2: Create Source Folder

- In the Explorer panel, right-click on your project folder
- Select "New Folder"
- Name it "src" (standard for source code)

Step 3: Create Package Structure (Optional)

- In the Explorer panel, right-click on the "src" folder
- Select "New Folder"
- Create a package path (e.g., "com/example/myapp")
- Note: Packages use directory structure to organize code

Project Structure

Recommended Structure

```
MyFirstJavaProject/  
├── src/  
│   ├── com/  
│   │   ├── example/  
│   │   │   └── myapp/  
│   │   │       └── Main.java  
├── bin/  
│   └── (compiled .class files)  
└── lib/  
    └── (external libraries)
```

- 📁 **src/** - Source code files (.java)
- 📁 **bin/** - Compiled bytecode (.class)
- 📁 **lib/** - External libraries (.jar)

Project Settings

Java Project Configuration

VS Code creates a .vscode folder with configuration files:

- 📄 **settings.json** - Project-specific settings
- 📄 **launch.json** - Debug configuration
- 📄 **tasks.json** - Build tasks

Classpath Configuration

If you need to add external libraries:

- Create a "lib" folder in your project
- Add .jar files to the lib folder
- Right-click on the .jar file
- Select "Add to Java Project Classpath"

Project Explorer and Navigation

Explorer View

- 📁 Shows file system structure
- 📁 Create, rename, delete files
- 📁 Organize files in folders

Java Projects View

- 🔗 Shows logical project structure
- 🔗 Organizes by packages
- 🔗 Shows dependencies

Outline View

- 👤 Shows structure of current file
- 👤 Lists classes, methods, fields
- 👤 Quick navigation within file



Now that you've created a Java project structure, you're ready to write your first Java program! In the next slide, we'll create a simple "Hello World" application.

Your First Java Program: Hello World

Creating Your First Java File

Step 1: Create a Java File

- Right-click on your source folder (src)
- Select "New File"
- Name it "HelloWorld.java"
- Note: Java class files must end with .java extension

Step 2: Using Package Structure (Optional)

- If using packages, create file in the appropriate folder
- Example: src/com/example/myapp/HelloWorld.java
- Add package declaration at the top of your file



In Java, the filename must match the class name exactly, including case. For a class named "HelloWorld", the file must be named "HelloWorld.java".

VS Code Features While Coding

Code Completion

IntelliSense suggests:

- Classes and interfaces
- Methods and fields
- Variables and keywords
- Trigger with Ctrl+Space

Code Snippets

Quick templates for common code:

- Type "sysout" → System.out.println()
- Type "main" → main method
- Type "class" → class definition
- Type "for" → for loop

Error Detection

Real-time error highlighting:

- Syntax errors
- Type mismatches
- Undefined variables
- Hover over errors for details

Code Actions

Quick fixes and refactoring:

- Click on lightbulb icon
- Or use Ctrl+. / Cmd+.
- Import missing classes
- Fix common errors

The Hello World Program

```
// If using a package
package com.example.myapp;

/**
 * A simple Hello World program
 */
public class HelloWorld {
    public static void main(String[] args) {
        // Print a message to the console
        System.out.println("Hello, World!");
    }
}
```

Code Explanation

- ✓ **package declaration:** Specifies the package this class belongs to
- ✓ **class declaration:** Defines a class named HelloWorld
- ✓ **main method:** Entry point for Java applications
- ✓ **System.out.println():** Prints text to the console

Understanding the Main Method

```
public static void main(String[] args) {
    // Code goes here
}
```

Main Method Breakdown

- ✓ **public:** Accessible from anywhere
- ✓ **static:** Can be called without creating an object
- ✓ **void:** Does not return any value
- ✓ **main:** Special method name recognized by JVM
- ✓ **String[] args:** Command-line arguments

Output Methods

System.out Methods

println()

Prints text and adds a new line

```
System.out.println("Hello!");
```

print()

Prints text without a new line

```
System.out.print("Hello ");
```

printf()

Formatted output

```
System.out.printf("Number: %d", 5);
```

format()

Returns formatted string

```
String.format("Pi: %.2f", 3.14159);
```



You can print variables by including them in your print statements:
System.out.println("Value: " + myVariable);

Try It Yourself

Challenge 1

Modify the Hello World program to print your name:

```
System.out.println("Hello, [Your Name]!");
```

Challenge 2

Print multiple lines of text:

```
System.out.println("Line 1");
System.out.println("Line 2");
System.out.println("Line 3");
```

Challenge 3

Print a simple pattern:

```
System.out.println("*");
System.out.println("***");
System.out.println("*****");
System.out.println("*****");
System.out.println("*****");
```



Now that you've created your first Java program, let's learn more about the basic structure of Java programs and how to compile and run them.

Basic Structure of a Java Program

Java Program Components

```
// Package declaration
package com.example.myapp;

// Import statements
import java.util.Scanner;
import java.util.ArrayList;

/**
 * Class documentation comment
 */
public class MyProgram {
    // Fields (class variables)
    private int count;
    public static final double PI = 3.14159;

    // Constructor
    public MyProgram() {
        count = 0;
    }

    // Methods
    public void incrementCount() {
        count++;
    }

    // Main method
    public static void main(String[] args) {
        // Local variables
        int x = 10;

        // Create object
        MyProgram program = new MyProgram();

        // Method calls
        program.incrementCount();

        // Output
        System.out.println("Hello, World!");
    }
}
```

Class Components

Fields (Variables)

Store data within a class

```
private int count;
public static final double PI = 3.14159;
```

Access modifiers: private, public, protected, default
Non-access modifiers: static, final, abstract, etc.

Constructors

Special methods that initialize objects

```
public MyProgram() {
    count = 0;
}
```

Characteristics: Same name as class, no return type

Methods

Define behavior of objects

```
public void incrementCount() {
    count++;
}
```

Components: Access modifier, return type, name, parameters, body

Key Components Explained

Package Declaration

Organizes classes into namespaces to avoid naming conflicts

```
package com.example.myapp;
```

Import Statements

Include classes from other packages

```
import java.util.Scanner;
```

Class Declaration

Blueprint for creating objects

```
public class MyProgram { ... }
```

Main Method

Entry point for Java applications

```
public static void main(String[] args) { ... }
```

Java Naming Conventions

Classes

- ✔ PascalCase (UpperCamelCase)
- ✔ Nouns or noun phrases
- ✔ Examples: Student, BankAccount

Methods

- ✔ camelCase
- ✔ Verbs or verb phrases
- ✔ Examples: calculateTotal(), getName()

Variables

- ✔ camelCase
- ✔ Meaningful names
- ✔ Examples: firstName, totalAmount

Constants

- ✔ UPPER_SNAKE_CASE
- ✔ All uppercase with underscores
- ✔ Examples: MAX_SIZE, PI

Java Comments

Single-line Comments

```
// This is a single-line comment
int x = 10; // Comment after code
```

Used for brief explanations or notes

Multi-line Comments

```
/* This is a multi-line comment
 * It can span multiple lines
 * Use it for longer explanations
 */
```

Used for longer explanations

Javadoc Comments

```
/**
 * This is a Javadoc comment
 * @param name The person's name
 * @return A greeting message
 */
```

Used for API documentation

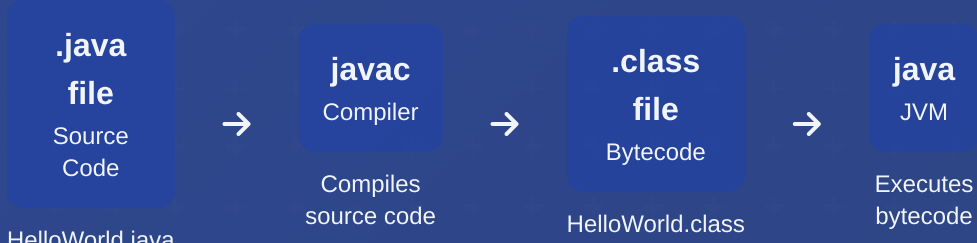


Understanding the basic structure of Java programs is essential for writing clean, maintainable code. As you continue learning Java, you'll become more familiar with these components and how they work together.

Compiling and Running Java Programs

The Java Compilation Process

From Source Code to Execution



Why Compilation?

- ✔ Converts human-readable code to machine-executable format
- ✔ Checks for syntax errors and type safety
- ✔ Bytecode is platform-independent (Write Once, Run Anywhere)
- ✔ JVM interprets bytecode for the specific operating system

Running Java Programs

Running Without Packages

```
# Compile
javac HelloWorld.java

# Run
java HelloWorld
```

Note: Do not include the .class extension when running

Running With Packages

```
# Compile from project root
javac com/example/myapp/HelloWorld.java

# Run from project root
java com.example.myapp.HelloWorld
```

Note: Use dots (.) in the package path when running

Compiling in VS Code

Method 1: Using VS Code Java Extension

1. Open your Java file in VS Code
2. Click the "Run" button above the main method
3. Or right-click in the editor and select "Run Java"
4. VS Code will automatically compile and run your program

Method 2: Using Terminal in VS Code

1. Open the integrated terminal (View → Terminal)
2. Navigate to your project directory
3. Compile with: `javac HelloWorld.java`
4. Run with: `java HelloWorld`

 When using packages, you need to compile and run from the root directory of your project, not from inside the package directory.

Command-line Arguments

```
# Run with arguments
java HelloWorld arg1 arg2 arg3

# Access in main method
public static void main(String[] args) {
    // args[0] is "arg1", args[1] is "arg2", etc.
}
```

VS Code Run Configurations

Creating a Run Configuration

1. Click on "Run and Debug" in the sidebar (or press Ctrl+Shift+D)
2. Click "create a launch.json file"
3. Select "Java"
4. Choose a configuration template (e.g., "Java: Current File")
5. VS Code creates a launch.json file in the .vscode folder

Sample launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "java",
      "name": "HelloWorld",
      "request": "launch",
      "mainClass": "com.example.myapp.HelloWorld",
      "projectName": "MyFirstJavaProject",
      "args": ["arg1", "arg2"]
    }
  ]
}
```

Using Run Configurations

- ✔ Select configuration from dropdown in Run and Debug panel
- ✔ Click the green play button or press F5
- ✔ Output appears in the Debug Console

Common Compilation Errors

Syntax Errors

```
// Missing semicolon
int x = 10

// Missing closing brace
public void method() {
    System.out.println("Hello");
// }
```

Errors in the language syntax

Semantic Errors

```
// Type mismatch
String s = 42;

// Undefined variable
int sum = x + y;

// Incompatible types
int[] arr = "Hello";
```

Errors in the program logic

Runtime Errors

```
// Division by zero
int result = 10 / 0;

// Null pointer exception
String s = null;
int length = s.length();

// Array index out of bounds
int[] arr = {1, 2, 3};
int value = arr[3];
```

Errors that occur during execution

 VS Code's Java extension provides real-time error detection and suggestions to help you fix compilation errors as you type. The Problems panel shows a list of all errors and warnings in your code.

Debugging Java Programs in VS Code

What is Debugging?

Definition

Debugging is the process of finding and fixing errors (bugs) in your code.

It allows you to:

- ✔ Pause program execution at specific points
- ✔ Inspect variable values during execution
- ✔ Step through code line by line
- ✔ Track the flow of execution
- ✔ Identify the root cause of errors

When to Debug?

- 🚩 When your program crashes with an exception
- 🚩 When your program produces unexpected results
- 🚩 When you want to understand how a piece of code works
- 🚩 When you're learning a new programming concept

Debugging Tools in VS Code

Breakpoints

Pause execution at specific lines:

- Click in the gutter (left margin)
- Or press F9 on the current line
- Red dot appears when set
- Program stops before executing that line

Watch Expressions

Monitor variable values:

- Add expressions in Watch panel
- Track complex expressions
- Updates as you step through code
- Right-click variables to add to watch

Call Stack

Track method execution:

- Shows current execution path
- Displays nested method calls
- Click to navigate between frames
- Helps understand program flow

Variables Panel

Inspect current state:

- Shows all variables in scope
- Organized by context
- Expand objects to see properties
- Updates as you step through code

Debug Actions

Continue (F5)	Pause	Stop (Shift+F5)
Restart	Step Into (F11)	Step Out (Shift+F11)
Step Over (F10)	Hot Code Replace	Toggle Breakpoint

Setting Up for Debugging

Step 1: Create a Launch Configuration

- Click on the Run and Debug icon in the sidebar
- Click "create a launch.json file"
- Select "Java" as the environment
- Choose a configuration template (e.g., "Java: Current File")

Step 2: Customize launch.json (Optional)

You can customize your debug configuration:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "java",
      "name": "Debug Main",
      "request": "launch",
      "mainClass": "com.example.myapp.Main",
      "args": ["arg1", "arg2"]
    }
  ]
}
```

💡 VS Code's Java extension often creates debug configurations automatically when you run your program for the first time.

Debugging Workflow

Step 1: Set Breakpoints

Place breakpoints at strategic locations:

- ✔ Before suspected problem areas
- ✔ At the beginning of important methods
- ✔ Inside conditional blocks

Step 2: Start Debugging

- Click the Run and Debug icon in the sidebar
- Select your debug configuration
- Click the green play button or press F5
- Program runs until it hits a breakpoint

Step 3: Analyze and Navigate

- ✔ **Step Over (F10):** Execute current line, don't dive into methods
- ✔ **Step Into (F11):** Dive into method calls
- ✔ **Step Out (Shift+F11):** Complete current method and return
- ✔ **Continue (F5):** Run until next breakpoint

Debugging Example

Sample Code with Bug

```
public class DebugDemo {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};
        int sum = calculateSum(numbers);
        double average = calculateAverage(numbers);
        System.out.println("Sum: " + sum);
        System.out.println("Average: " + average);
    }

    public static int calculateSum(int[] arr) {
        int sum = 0;
        // Bug: loop starts from 1 instead of 0
        for (int i = 1; i < arr.length; i++) {
            sum += arr[i];
        }
        return sum;
    }

    public static double calculateAverage(int[] arr) {
        int sum = calculateSum(arr);
        // Bug: using length without checking if it's zero
        return sum / arr.length;
    }
}
```

Debugging Steps

- Set breakpoints at lines 4, 12, and 19
- Start debugging with F5
- When stopped at line 4, examine the numbers array
- Step into calculateSum() with F11
- Notice the loop starts at i=1 (skipping first element)
- Step through the loop, watching sum value
- Continue to calculateAverage()
- Notice integer division truncates decimal part

Fixing the Bugs

```
// Fix 1: Start loop from index 0
for (int i = 0; i < arr.length; i++) {
    sum += arr[i];
}
```

```
// Fix 2: Cast to double for proper division
return sum / (double)arr.length;
```

After fixing, the program correctly calculates sum=15 and average=3.0



Debugging is an essential skill for every programmer. With practice, you'll become more efficient at finding and fixing bugs in your code. VS Code's debugging tools make this process much easier and more intuitive.

Basic Java Syntax

Variables and Data Types

Primitive Data Types

Integer Types:

byte: 8 bits (-128 to 127)
short: 16 bits (-32,768 to 32,767)
int: 32 bits (±2 billion)
long: 64 bits (±9 quintillion)

Other Types:

float: 32-bit floating point
double: 64-bit floating point
char: 16-bit Unicode character
boolean: true or false

Variable Declaration

```
// Declaration
int age;
double salary;
boolean isStudent;

// Declaration with initialization
int age = 25;
double salary = 50000.50;
boolean isStudent = true;
char grade = 'A';
String name = "John";
```

Reference Types

```
// String (most common reference type)
String message = "Hello, World!";

// Arrays
int[] numbers = {1, 2, 3, 4, 5};
String[] names = new String[3];

// Objects
Scanner scanner = new Scanner(System.in);
ArrayList<String> list = new ArrayList<>();
```

Operators

Arithmetic Operators

+ Addition
- Subtraction
* Multiplication
/ Division
% Modulus (remainder)
++ Increment
-- Decrement

Comparison Operators

== Equal to
!= Not equal to
> Greater than
< Less than
>= Greater than or equal to
<= Less than or equal to

Logical Operators

&& Logical AND
|| Logical OR
! Logical NOT

Assignment Operators

= Assignment
+= Add and assign
-= Subtract and assign
*= Multiply and assign
/= Divide and assign
%= Modulus and assign

Control Flow Statements

If-Else Statements

```
// Simple if
if (age >= 18) {
    System.out.println("You are an adult.");
}

// If-else
if (score >= 60) {
    System.out.println("You passed!");
} else {
    System.out.println("You failed.");
}

// If-else-if ladder
if (grade >= 90) {
    System.out.println("A");
} else if (grade >= 80) {
    System.out.println("B");
} else if (grade >= 70) {
    System.out.println("C");
} else {
    System.out.println("F");
}
```

Switch Statement

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    // ... more cases
    default:
        System.out.println("Invalid day");
}
```

Loops

```
// For loop
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

// While loop
int j = 0;
while (j < 5) {
    System.out.println(j);
    j++;
}

// Do-while loop
int k = 0;
do {
    System.out.println(k);
    k++;
} while (k < 5);

// For-each loop (enhanced for loop)
int[] numbers = {1, 2, 3, 4, 5};
for (int num : numbers) {
    System.out.println(num);
}
```

String Operations

String Creation

```
// String literals
String name = "John";

// Using new keyword
String city = new String("New York");

// String concatenation
String greeting = "Hello, " + name + "!";
```

Common String Methods

length() - Get string length	toUpperCase() - Convert to uppercase
charAt(index) - Get character at position	trim() - Remove leading/trailing spaces
substring(start, end) - Extract substring	replace(old, new) - Replace text
indexOf(str) - Find position of substring	split(delimiter) - Split into array
toLowerCase() - Convert to lowercase	equals(str) - Compare content

```
String text = "Hello, World!";
int length = text.length(); // 13
char firstChar = text.charAt(0); // 'H'
String sub = text.substring(0, 5); // "Hello"
boolean contains = text.contains("World"); // true
```

Arrays

Array Declaration

```
// Declaration
int[] numbers;

// Declaration with size
int[] scores = new int[5];

// Declaration with initialization
int[] values = {10, 20, 30, 40, 50};
```

Accessing Elements

```
int[] numbers = {10, 20, 30, 40, 50};

// Get element (zero-based indexing)
int first = numbers[0]; // 10
int third = numbers[2]; // 30

// Set element
numbers[1] = 25; // Changes 20 to 25

// Array length
int length = numbers.length; // 5
```

Iterating Arrays

```
int[] numbers = {10, 20, 30, 40, 50};

// Using for loop
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}

// Using enhanced for loop
for (int num : numbers) {
    System.out.println(num);
}
```

 This slide covers the fundamental syntax elements of Java. As you continue learning, you'll build on these basics to create more complex and powerful programs. Practice is key to mastering these concepts!

Java Comments

Types of Comments

Single-line Comments

```
// This is a single-line comment

int age = 25; // This is an end-of-line comment

// You can also use multiple single-line comments
// to create a block of comments
// like this
```

Single-line comments start with `//` and continue until the end of the line.

Multi-line Comments

```
/* This is a multi-line comment.
 * It can span multiple lines.
 * The asterisks at the beginning of each line
 * are not required, but they improve readability.
 */

int total = 100; /* This is an inline
multi-line comment */ int count = 5;
```

Multi-line comments start with `/*` and end with `*/`.

Javadoc Comments

```
/**
 * This is a Javadoc comment.
 * It is used to generate documentation.
 *
 * @param name The name of the person
 * @return A greeting message
 * @throws IllegalArgumentException If name is empty
 */
public String greet(String name) {
    if (name.isEmpty()) {
        throw new IllegalArgumentException("Name cannot be empty");
    }
    return "Hello, " + name + "!";
}
```

Javadoc comments start with `/**` and end with `*/`. They are used to generate API documentation.

Javadoc Tags

Method Tags

`@param` Describes a method parameter
`@return` Describes the return value
`@throws` Describes an exception
`@see` Provides a reference to another element

Class Tags

`@author` Specifies the author
`@version` Specifies the version
`@since` Specifies when added
`@deprecated` Marks as deprecated

Example Class with Javadoc

```
/**
 * Represents a student in a school system.
 *
 * @author Jane Doe
 * @version 1.0
 * @since 2023-01-01
 */
public class Student {
    private String name;
    private int age;

    /**
     * Creates a new Student with the given name and age.
     *
     * @param name The student's name
     * @param age The student's age
     * @throws IllegalArgumentException If age is negative
     */
    public Student(String name, int age) {
        // Constructor implementation
    }
}
```

Comment Best Practices

Do's

- ✔ Comment complex logic or algorithms
- ✔ Use Javadoc for public methods and classes
- ✔ Explain "why" rather than "what" the code does
- ✔ Keep comments up-to-date with code changes
- ✔ Use comments to mark TODOs or future improvements

Don'ts

- ✗ Don't state the obvious in comments
- ✗ Avoid commented-out code in production
- ✗ Don't write misleading or outdated comments
- ✗ Avoid excessive comments that clutter code
- ✗ Don't use comments for version control

Comments in VS Code

Keyboard Shortcuts

-  **Ctrl+I** - Toggle line comment
-  **Shift+Alt+A** - Toggle block comment
-  **/** + Enter** - Generate Javadoc

Javadoc Generation

VS Code can automatically generate Javadoc templates:

- Position cursor above method/class
- Type `/**` and press Enter
- VS Code generates template with parameters
- Fill in the descriptions

Comment Navigation

-  Use Outline view to see Javadoc comments
-  Hover over methods to see Javadoc
-  Use TODO comments for task tracking
-  Search for TODO comments in Problems panel

 Comments are an essential part of writing maintainable code. They help other developers (and your future self) understand your code's purpose and functionality. Well-commented code is easier to maintain, debug, and extend.

Common Errors and Troubleshooting

Compilation Errors

These errors occur when the Java compiler cannot understand your code. VS Code will usually highlight these directly.

Syntax Errors

- ✖ Missing semicolons (;)
- ✖ Mismatched parentheses (), braces {}, or brackets []
- ✖ Incorrect keywords (e.g., pubic instead of public)
- ✖ Undeclared variables
- ✖ Type mismatches (e.g., assigning a String to an int)

```
// Example: Missing semicolon int x = 5 // Error: ';' expected
```

Class Not Found / Symbol Not Found

- ✖ Trying to use a class that hasn't been imported
- ✖ Misspelled class, method, or variable names
- ✖ Incorrect file name (must match public class name)

```
// Example: Using Scanner without import Scanner input = new Scanner(System.in); // Error: Scanner cannot be resolved to a type
```

Runtime Errors (Exceptions)

These errors occur while your program is running. They often cause the program to crash if not handled.

NullPointerException

Trying to use an object reference that is null.

```
String name = null;
System.out.println(name.length()); // Throws NullPointerException
```

ArrayIndexOutOfBoundsException

Trying to access an array element with an invalid index (e.g., negative or too large).

```
int[] numbers = {1, 2, 3};
System.out.println(numbers[3]); // Throws
ArrayIndexOutOfBoundsException
```

ArithmeticException

An arithmetic error occurs, like division by zero.

```
int result = 10 / 0; // Throws ArithmeticException: / by zero
```

InputMismatchException

When using Scanner, if the input type doesn't match the expected type (e.g., entering text for nextInt()).

```
Scanner input = new Scanner(System.in);
System.out.print("Enter an integer: ");
int num = input.nextInt(); // Throws InputMismatchException if
non-integer is entered
```

Troubleshooting Tips

🔍 Read Error Messages Carefully

Error messages provide clues: type of error, line number, and sometimes a description.

🔧 Use the Debugger

Step through your code, inspect variables, and understand the execution flow.

🖨️ Print Statements (System.out.println)

Insert print statements to check variable values at different points in your code (a simpler form of debugging).

🔄 Isolate the Problem

Comment out sections of code to narrow down where the error is occurring.

📖 Check Documentation / Online Resources

Look up error messages or consult Java documentation (e.g., Oracle, Stack Overflow).

🔄 Double-Check Logic

Sometimes the code is syntactically correct but logically flawed. Dry run your code manually.

☕ Take a Break

Stepping away can often help you see the problem with fresh eyes.

VS Code Error Highlighting

- 💡 VS Code underlines errors in red (squiggles).
- 💡 Hover over the squiggle to see the error message.
- 💡 The "Problems" panel (View > Problems) lists all errors and warnings.
- 💡 Pay attention to warnings (yellow squiggles) as they can indicate potential issues.

🛡️ Encountering errors is a normal part of programming. Learning to effectively troubleshoot and debug is a crucial skill for any developer. Don't get discouraged by errors; see them as learning opportunities!

Summary and Next Steps

Key Takeaways

- ★ Java is a powerful, platform-independent language.
- ★ JDK, JRE, and JVM are core to Java development.
- ★ VS Code is a versatile IDE for Java with useful extensions.
- ★ Your first "Hello, World!" program is a great start!
- ★ Understanding basic syntax, compilation, and debugging is crucial.
- ★ Comments make your code understandable.
- ★ Errors are part of learning; debugging is a key skill.

Next Steps

- 📖 **Practice Regularly:** Write small programs daily.
- 📖 **Explore Java Basics:** Dive deeper into variables, operators, control flow.
- 📖 **Object-Oriented Programming (OOP):** Learn about classes, objects, inheritance, polymorphism.
- 📖 **Data Structures:** Understand arrays, lists, maps, etc.
- 📖 **Build Simple Projects:** E.g., a calculator, a to-do list app.
- 📖 **Read Code:** Explore open-source Java projects on GitHub.
- 📖 **Join Communities:** Engage in forums like Stack Overflow, Reddit (r/java).



Your Java journey has just begun! Keep learning, keep coding, and enjoy the process!

Questions & Answers

Q: What if I get a "java: command not found" error?

A: This means Java is not properly installed or not in your system PATH. Reinstall the JDK and ensure you select "Add to PATH" during installation, or manually add the Java bin directory to your system PATH.

Q: Why doesn't VS Code recognize my Java files?

A: Make sure you have installed the Extension Pack for Java. Also, ensure your .java files are in a proper project structure with a src folder, and that VS Code has opened the correct project folder.

Q: What's the difference between JDK, JRE, and JVM?

A: JVM runs Java programs, JRE includes JVM plus libraries for running Java apps, and JDK includes JRE plus development tools like compiler. For development, you need the full JDK.

Q: Can I use other IDEs instead of VS Code?

A: Yes! Popular alternatives include IntelliJ IDEA, Eclipse, and NetBeans. VS Code is lightweight and beginner-friendly, but you can choose any IDE that suits your preferences.

Q: How do I fix compilation errors in my first program?

A: Common issues include: incorrect class name (must match filename), missing semicolons, wrong capitalization (Java is case-sensitive), or missing curly braces. Check the error message carefully for clues.

Thank You!

Happy Coding with Java! 🚀